

Even More Pizza

Group_G

Artur Miguel Pinheiro Hargreaves Santoalha (up202412309)

Daniel Gomes Mendes (up202503473)

Miguel Luís Braga Eiras (up202205107)

Inteligência Artificial FEUP-MECD06-2025/2026-2S

(1) Introduction

- Constraint optimization problems (COP) are problems with constraints and preferences, with the main goal of getting the optimal solution to our objective function
- There are multiple types of COP, namely the shortest path problem, the assignment problem, and the set covering problem.
- In many of these problems, the optimal solution is complex in both time and memory requirements
- Therefore, the use of local search and meta-heuristics is crucial to obtain locally optimal solutions to these problems, at a much lower computational load.

(2) Definition of optimization problem to be solved

- The chosen problem is Even More Pizza (Topic 3B), a Google Hash Code challenge; it is composed of five different sets of pizzas with a variable number of ingredients, to be delivered to teams of 4,3 and 2 people:
 - a_example: small number of pizzas, ingredients, and teams
 - b_little_bit_of_everything: moderate number of pizzas, ingredients, and teams
 - c_many_ingredients: large number of pizzas, ingredients, and teams
 - d_many_pizzas: very large number of pizzas, ingredients, and teams
 - e_many_teams: extremely large number of pizzas, ingredients, and teams
- This problem is an instance of the **Set Covering Problem**, more precisely a **Maximum Coverage Problem** (trying to cover as many unique ingredients as possible with limited subsets/pizzas)
- The approach will be to implement a greedy local search algorithm to get a preliminary solution, and then improve this solution by applying the following meta-heuristics:
 - Tabu search
 - Simulated annealing
 - Genetic Algorithms

(3) Related work

- [1] Alireza Eydi and Javad Mohebi, “Modeling and solution of maximal covering problem considering gradual coverage with variable radius over multi-periods,” *RAIRO - Operations Research*, vol. 52, no. 4–5, pp. 1245–1260, Apr. 2018
- [2] Y. N. Hoogendoorn, “The Maximum Coverage Problem Investigating Approximation Algorithms for Variants of the Maximum Coverage Problem,” Erasmus University Rotterdam Erasmus School of Economics, 2017. Accessed: Mar. 03, 2026. [Online]. Available: <https://thesis.eur.nl/pub/38550/Hoogendoorn.pdf>
- [3] S. Atta, P. R. S. Mahapatra, and A. Mukhopadhyay, “Solving maximal covering location problem using genetic algorithm with local refinement,” *Soft Computing*, vol. 22, no. 12, pp. 3891–3906, Apr. 2017
- [4] S. Russel and P. Norvig, *Artificial intelligence: A Modern approach*, 4th ed. Prentice Hall, 2021.
- [5] “Google Hashcode 2021 Practice Round Discussion - Codeforces,” *Codeforces*, 2021. <https://codeforces.com/blog/entry/87699> (accessed Feb. 27, 2026).
- [6] Z-G. Ren, Z-R. Feng, L-J. Ke, and Z-J. Zhang, “New ideas for applying ant colony optimization to the set covering problem,” *Computers & Industrial Engineering*, vol. 58, no. 4, pp. 774–784, May 2010

(4) Formulation of the problem as optimization problem

COP Model Formulation

To solve this mathematically, we define the following components based on the problem description:

Variables and Domains

M : Total available pizzas $P_0, P_1, \dots, P_{M-1}$.

T_N : Number of teams of size $N \in 2, 3, 4$.

Decision Variable ($x_{i,j,N}$): A binary variable where $x_{i,j,N} = 1$ if Pizza i is assigned to a Team j with N members, and 0 otherwise.

D : Total number of deliveries

Constraints

Pizza Uniqueness: Each pizza can be delivered to at most one team.

$$\sum_j x_{i,j} \leq 1 \quad \forall i \in 0, \dots, M-1$$

Team Capacity: If a team j of size N receives a delivery, the team must receive exactly N pizzas.

$$\sum_i x_{i,j,N} \in 0, N \quad \forall \text{ Team } j \text{ of size } N$$

Team Count:

The Total number of deliveries to teams cannot exceed the Total number of Teams

$$D \leq \sum_N T_N$$

The number of deliveries to teams of size N cannot exceed T_N .

$$D_N \leq T_N$$

Objective Function

The goal is to maximize the sum of the squares of the number of unique ingredients per team. Let I_j be the set of unique ingredients in all pizzas assigned to team j .

$$\text{Maximize } \sum_j (|I_j|)^2$$

(5) Meta-heuristics

- **Genetic Algorithms:** This meta-heuristic belongs to the family of evolutionary algorithms; uses concepts from genetics such as mutations, cross-over, and fitness over a number of generations to ensure the best fitting solutions prevail
- **Tabu Search:** A variant of the hill-climbing algorithm that maintains a list of tabu solutions, which are previously obtained solutions that cannot be revisited for a specific time (tabu tenure), preventing entrapment in some local extrema
- **Simulated Annealing:** A meta-heuristic that combines the principles of the hill-climbing algorithm with a random walk, ensuring that the algorithm does not get trapped in local extrema (in this case local maxima)
- **Ant Colony Optimization** (not implemented): A swarm intelligence algorithm where “ants” search for the minimum cost path in a graph, while communicating with each other through “pheromone” trails, which highlight the best path for all ants to follow

(6) GA - Implementation details

- **Generations:** defined as input
- **Population size:** defined as input
- **Initial Population:**
 - initialized by repeating the solution generated by local search
 - initialized by calculating multiple sols. generated by local search with random shuffle
- **Selection:** Select the top 25% of the population (not configurable)
- **Crossover:** Take half of deliveries from Parent 1 and Fill remaining from Parent 2 if pizzas are available (not configurable)
- **Mutation:** Swap one pizza between two deliveries of same size (not conf.)
- **Encoding :** Same format as deliveries
- **Fitness Function:** same as objective function, best score.
- **Python + Jupyter Notebook**

(6) TS - Implementation details

- **Iterations:** defined as input parameter (e.g., 500 or 1000)
- **Tabu List Size:** defined as input parameter (e.g., 25, 50, 100)
- **Initial Solution:** initialized using the greedy heuristic solution
- **Neighbourhood Generation:** Two move operators selected randomly
 - **Swap:** exchange one pizza between two deliveries
 - **Replace:** replace a pizza in a delivery with an unused pizza
- **Tabu Mechanism:** recent moves stored in a FIFO tabu list and moves in tabu list cannot be repeated unless they improve the best score
- **Aspiration Criterion:** tabu move accepted if it improves the global best solution
- **Fitness Function:** same as objective function, best score (I^2)
- **Encoding:** same format as deliveries (team size + list of pizza IDs)
- **Python + Jupyter Notebook**

(6) SA - Implementation details

- **Initial Temperature (T_{max}):** defined as input (e.g., 100 or 1000)
- **Minimum Temperature (T_{min}):** defined as input (e.g., 0.1)
- **Cooling Rate:** defined as input (e.g., 0.95 or 0.90)
- **Iterations per Temperature:** defined as input parameter
- **Initial Solution:** initialized using the greedy heuristic solution
- **Neighbourhood Generation:** randomly replace one pizza in a delivery with an unused pizza
- **Acceptance Rule:** always accept better solutions; worse solutions accepted with a given probability:
- **Cooling Schedule:** temperature decreases geometrically
- **Fitness Function:** same as objective function, best score (I^2)
- **Encoding:** same format as deliveries
- **Python + Jupyter Notebook**

(7) Experimental results

- Ran different parameterizations of the different algorithms
- Applied to datasets a,b,c due to time complexity (although also applicable to datasets d and e, these take significantly longer)
- Applied Mann-Whitney U rank test (95% confidence interval)

(7) Runs executed per input

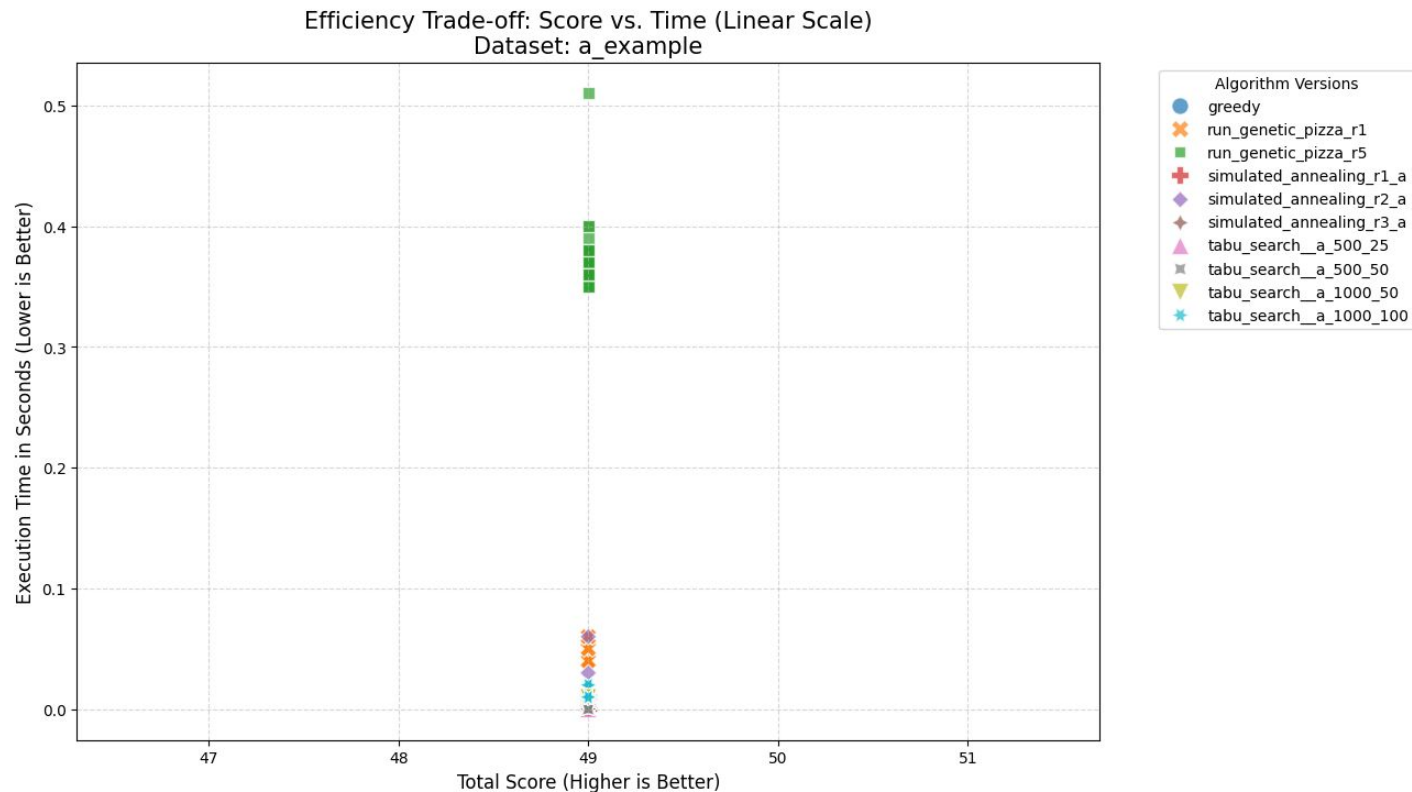
Input	Version	Parameters
a_example	greedy	NA
	run_genetic_pizza_r1	GEN=100 POP=100
	run_genetic_pizza_r5	GEN=1000 POP=100
	simulated_annealing_r1_a	TM=100 Tm=0.1 CR=0.95,IT=3
	simulated_annealing_r2_a	TM=1000 Tm=0.1 CR=0.95,IT=5
	simulated_annealing_r3_a	TM=1000 Tm=0.1 CR=0.90,IT=5
	tabu_search__a_1000_100	1000 100
	tabu_search__a_1000_50	1000 50
	tabu_search__a_500_25	500 25
b_little_bit_of_everything.in	tabu_search__a_500_50	500 50
	greedy	NA
	run_genetic_pizza_r2	GEN=100 POP=100
	run_genetic_pizza_r6	GEN=1000 POP=100
	simulated_annealing_r1_b	TM=100 Tm=0.1 CR=0.95,IT=3
	simulated_annealing_r2_b	TM=1000 Tm=0.1 CR=0.95,IT=5
	simulated_annealing_r3_b	TM=1000 Tm=0.1 CR=0.90,IT=5
	tabu_search__b_1000_100	1000 100
	tabu_search__b_1000_50	1000 50
c_many_ingredients.in	tabu_search__b_500_25	500 25
	tabu_search__b_500_50	500 50
	greedy	NA
	run_genetic_pizza_r3	GEN=100 POP=100
	run_genetic_pizza_r7	GEN=1000 POP=100
	simulated_annealing_r1_c	TM=100 Tm=0.1 CR=0.95,IT=3
	simulated_annealing_r2_c	TM=1000 Tm=0.1 CR=0.95,IT=5
	simulated_annealing_r3_c	TM=1000 Tm=0.1 CR=0.90,IT=5
	tabu_search__c_500_25	500 25
	tabu_search__c_500_50	500 50

(7) Experimental Results

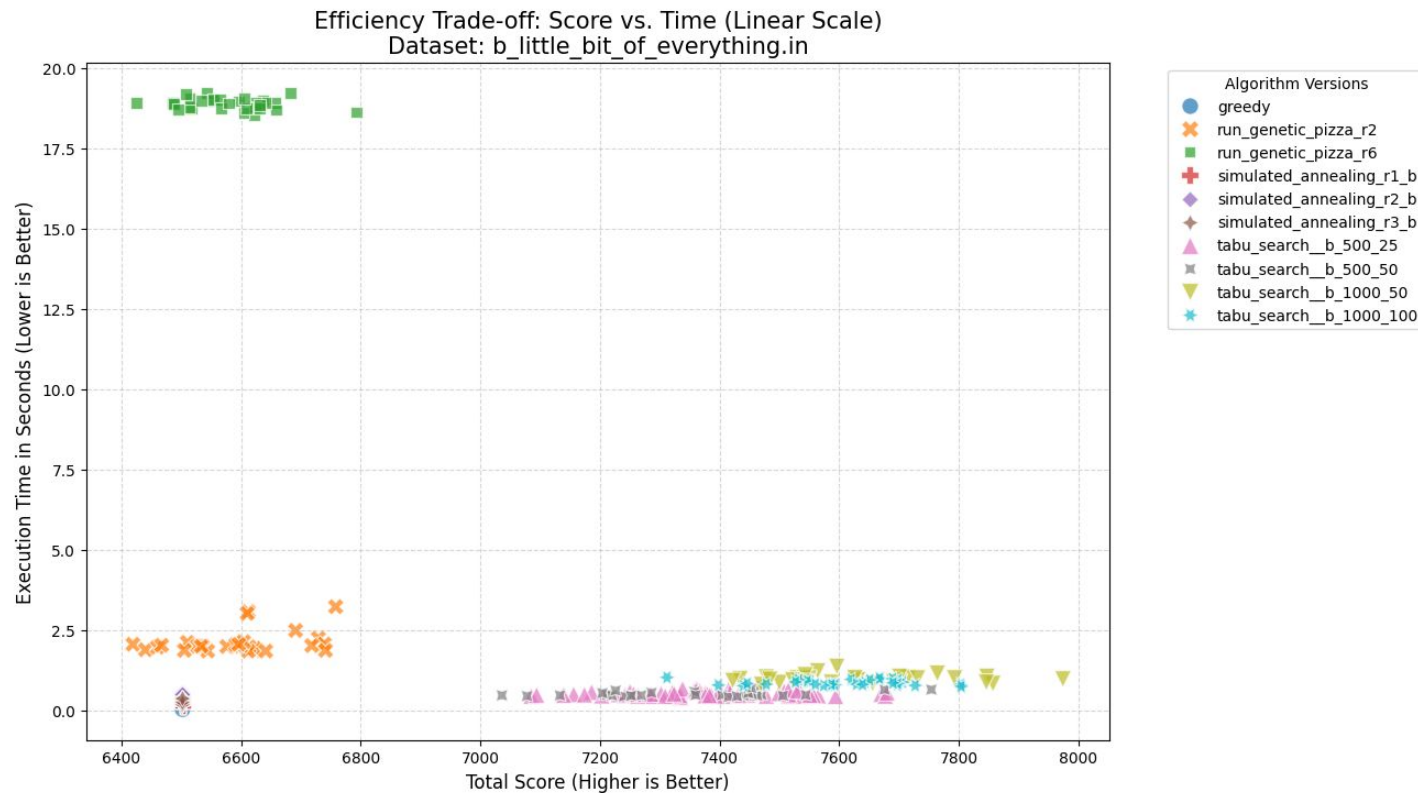
Table 1: Greedy local search results

Dataset	Score	Time
a	49	Less than 1 second
b	6502	Less than 1 second
c	229352265	4 minutes
d	2046959	27 minutes
e	8257534	1 hour

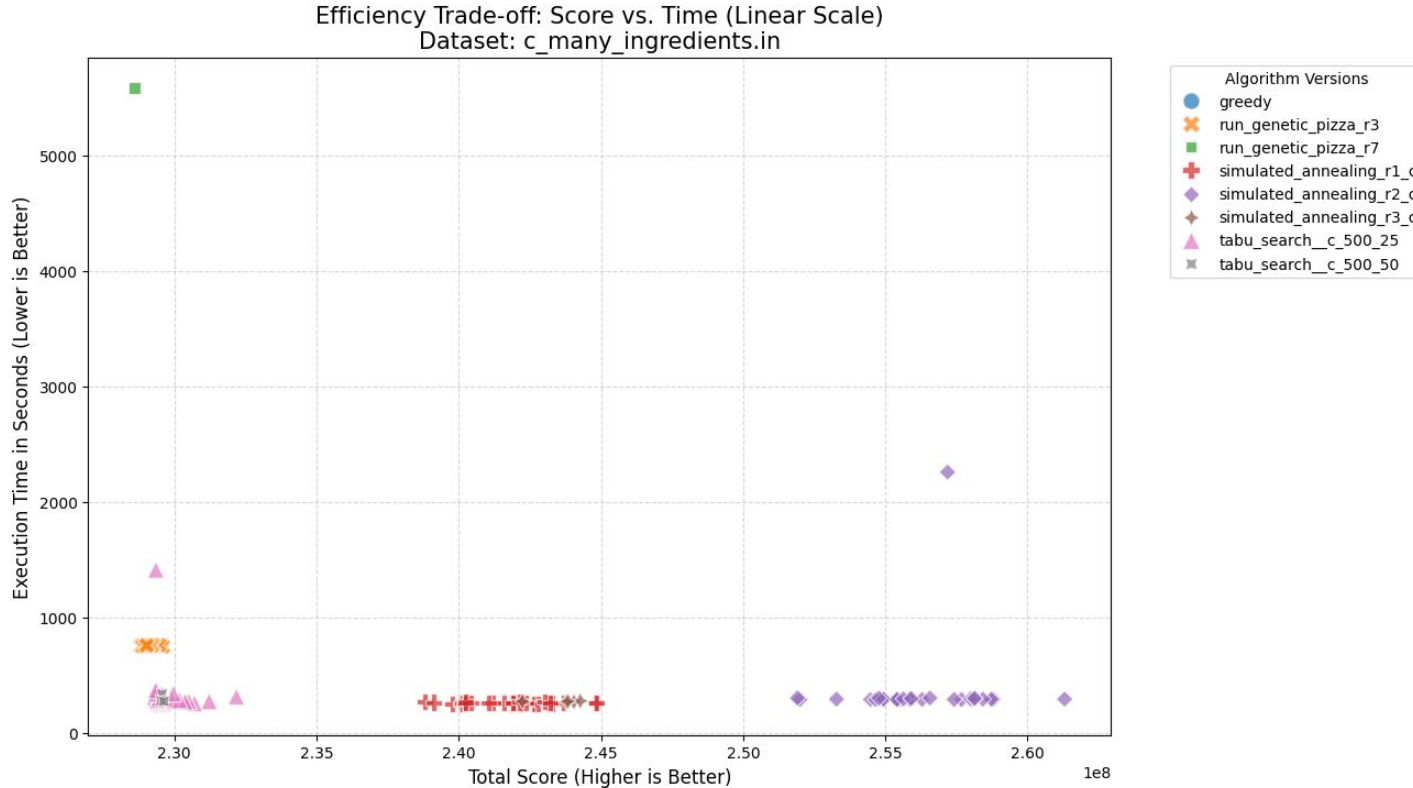
Distribution per algorithm: a_example



Distribution per algorithm: b_little_bit_of_everything.in



Distribution per algorithm: c_many_ingredients.in



(8) Mann-Whitney U rank test (95% confidence interval)

Data set	Algorithm run comparison	p-value	Statistically significant difference
a	run_genetic_pizza_r1 vs simulated_annealing_r1_a	1	No
a	run_genetic_pizza_r1 vs tabu_search__a_500_25	1	No
a	simulated_annealing_r1_a vs tabu_search__a_500_25	1	No
b	run_genetic_pizza_r2 vs simulated_annealing_r3_b	0.000000191222659	Yes
b	run_genetic_pizza_r2 vs tabu_search__b_1000_50	0.000000000030104	Yes
b	simulated_annealing_r3_b vs tabu_search__b_1000_50	0.00000000001209	Yes
c	simulated_annealing_r3_b vs tabu_search__b_1000_50	0.000000000030199	Yes
c	run_genetic_pizza_r3 vs tabu_search__c_500_25	0.000000007735613	Yes
c	simulated_annealing_r2_c vs tabu_search__c_500_25	0.000000000030010	Yes

(9) Conclusions

- For dataset a, all algorithms return the same score; for dataset b, the best choice of algorithm is tabu search (1000 iterations and 50 tabu size); for dataset c, the best choice is simulated annealing (temperature between 1000 and 0.1, cooling rate of 0.95 and 5 iterations per temperature);
- There are statistically significant differences between the algorithms for datasets b and c but none for dataset a;
- There is a trade-off between better solutions and lower execution times; this highlights the fact that, depending on the problem and available resources, one should always consider different algorithms to obtain an acceptable solution in short time.

Thank you for your attention!